Inventory Management Platform

End-to-End Implementation

A minimal but complete inventory management system with FastAPI, SQLAlchemy, async external data enrichment, and decorators for caching and logging.

Project Structure

inventory_platform/

├── app/

│ ├── __init__.py

│ ├── main.py

│ ├── models.py

│ ├── schemas.py

│ ├── database.py

│ ├── crud.py

│ ├── dependencies.py

```
│   ├── decorators.py
│   ├── external.py
│   └── routers/
│       ├── __init__.py
│       ├── products.py
│       └── inventory.py
├── tests/
│   ├── __init__.py
│   ├── test_products.py
│   └── test_inventory.py
├── requirements.txt
└── .env
```

Command to create folder structure

mkdir app\routers & mkdir tests & type nul > app__init__.py & type nul > app\main.py & type nul > app\models.py & type nul > app\schemas.py & type nul > app\database.py & type nul > app\crud.py & type nul > app\dependencies.py & type nul > app\decorators.py & type nul > app\external.py & type nul > app\routers__init__.py & type nul > app\routers\products.py & type nul > app\routers\inventory.py & type nul > tests__init__.py & type nul > tests\test_products.py & type nul > tests\test_inventory.py & type nul > requirements.txt & type nul > .env

---

Requirements File

requirements.txt

```
fastapi==0.115.6
uvicorn==0.34.0
sqlalchemy==2.0.36
pydantic==2.10.4
pydantic-core==2.27.2
python-dotenv==1.0.1
httpx==0.27.2
pytest==8.3.4
pytest-asyncio==0.24.0
```

Environment Configuration

.env

```
"DATABASE_URL=sqlite:///./inventory.db"
"EXTERNAL_API_URL=https://fakestoreapi.com/products"
"EXTERNAL_API_KEY=demo-key-not-required"
CACHE_TTL=300
LOG_LEVEL=INFO
```

Core Application Files

1. Database Configuration (`database.py`)

```python
# app/database.py

"""

Database configuration and session management.

"""

from sqlalchemy import create_engine

from sqlalchemy.ext.declarative import declarative_base

from sqlalchemy.orm import sessionmaker

import os

from dotenv import load_dotenv

load_dotenv()

# Database URL from environment or default to SQLite

DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///./inventory.db")

# Create engine with SQLite-specific configuration
```

```
engine = create_engine(
    DATABASE_URL,
    connect_args={"check_same_thread": False} if "sqlite" in DATABASE_URL else {}
)
# Session factory
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
# Base class for models
Base = declarative_base()
def get_db():
    """
    Dependency for database session.
    Yields a session and ensures it's closed after use.
    """
    db = SessionLocal()
```

```
    try:
        yield db
    finally:
        db.close()
```

---

2. Database Models (`models.py`)

```
# app/models.py
"""
SQLAlchemy ORM models.
"""
from sqlalchemy import Column, Integer, String, Float, DateTime, Boolean, Text
from sqlalchemy.sql import func
from app.database import Base
```

```python
class Product(Base):
    """Product model representing inventory items."""

    __tablename__ = "products"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(200), nullable=False, index=True)
    description = Column(Text, nullable=True)
    price = Column(Float, nullable=False)
    stock = Column(Integer, nullable=False, default=0)
    category = Column(String(100), nullable=True)
    sku = Column(String(50), unique=True, index=True, nullable=False)
    is_active = Column(Boolean, default=True)
    external_id = Column(String(100), nullable=True)  # For external enrichment
```

```
    external_data = Column(Text, nullable=True)  # JSON string from external API

    created_at = Column(DateTime, server_default=func.now())

    updated_at = Column(DateTime, onupdate=func.now())



    def __repr__(self):

        return f"<Product(id={self.id}, name='{self.name}', stock={self.stock})>"
```

---

3. Pydantic Schemas (`schemas.py`)

```
# app/schemas.py

"""

Pydantic models for request/response validation.

"""

from pydantic import BaseModel, Field, validator

from typing import Optional, Dict, Any
```

```
from datetime import datetime


# ===== Base Schemas =====

class ProductBase(BaseModel):

    """Base product schema."""

    name: str = Field(..., min_length=1, max_length=200)

    description: Optional[str] = None

    price: float = Field(..., gt=0)

    stock: int = Field(0, ge=0)

    category: Optional[str] = Field(None, max_length=100)

    sku: str = Field(..., min_length=1, max_length=50)



    @validator('sku')

    def validate_sku(cls, v):

        """Validate SKU format (uppercase, no spaces)."""
```

```python
        if ' ' in v:
            raise ValueError('SKU must not contain spaces')
        return v.upper()

class ProductCreate(ProductBase):
    """Schema for creating a product."""
    pass

class ProductUpdate(BaseModel):
    """Schema for updating a product (all fields optional)."""
    name: Optional[str] = Field(None, min_length=1, max_length=200)
    description: Optional[str] = None
    price: Optional[float] = Field(None, gt=0)
    stock: Optional[int] = Field(None, ge=0)
    category: Optional[str] = Field(None, max_length=100)
```

```
    is_active: Optional[bool] = None

    sku: Optional[str] = None

    @validator('sku', always=True)

    def validate_sku_update(cls, v):

        if v is not None:

            if ' ' in v:

                raise ValueError('SKU must not contain spaces')

            return v.upper()

        return v


# ===== Response Schemas =====

class ProductResponse(ProductBase):

    """Schema for product response."""

    id: int

    is_active: bool
```

```
    external_id: Optional[str]

    external_data: Optional[Dict[str, Any]]

    created_at: datetime

    updated_at: Optional[datetime]



    class Config:

        from_attributes = True



    @validator('external_data', pre=True)

    def parse_external_data(cls, v):

        """Parse JSON string to dict if needed."""

        if isinstance(v, str):

            try:

                import json

                return json.loads(v)
```

```
            except:
                return None
        return v


class ProductListResponse(BaseModel):
    """Schema for product list response with pagination."""
    items: list[ProductResponse]
    total: int
    page: int
    page_size: int
    pages: int


# ===== Enrichment Schemas =====
class ProductEnrichmentResponse(BaseModel):
    """Schema for enriched product data."""
```

```
    id: int
    name: str
    enriched_data: Dict[str, Any]
    enriched_at: datetime


class BatchEnrichmentResponse(BaseModel):
    """Schema for batch enrichment response."""
    total: int
    enriched: int
    failed: int
    details: list[Dict[str, Any]]
```

---

4. Decorators (`decorators.py`)

```python
# app/decorators.py

"""

Custom decorators for logging, caching, and performance monitoring.

"""

import functools

import time

import json

import hashlib

from typing import Any, Callable, Dict, Optional

from datetime import datetime, timedelta

import logging

logger = logging.getLogger(__name__)

# ===== Simple In-Memory Cache =====
```

```
class Cache:

    """Simple in-memory cache with TTL."""



    def __init__(self):

        self._cache: Dict[str, tuple[Any, datetime]] = {}



    def get(self, key: str) -> Optional[Any]:

        """Get cached value if not expired."""

        if key in self._cache:

            value, expires_at = self._cache[key]

            if datetime.now() < expires_at:

                return value

            else:

                del self._cache[key]
```

```
        return None


    def set(self, key: str, value: Any, ttl_seconds: int = 300):
        """Store value with TTL."""
        expires_at = datetime.now() + timedelta(seconds=ttl_seconds)
        self._cache[key] = (value, expires_at)


    def clear(self):
        """Clear all cache entries."""
        self._cache.clear()


    def invalidate(self, pattern: str):
        """Invalidate cache entries matching a pattern."""
        to_delete = [k for k in self._cache.keys() if pattern in k]
        for k in to_delete:
```

```
            del self._cache[k]


# Global cache instance

cache = Cache()


# ===== Logging Decorator =====

def log_execution(func: Callable) -> Callable:

    """

    Decorator to log function execution with arguments and return value.

    """

    @functools.wraps(func)

    def wrapper(*args, **kwargs):

        # Prepare log data

        func_name = func.__name__

        args_str = ', '.join([repr(a) for a in args[1:]])  # Skip self
```

```
        kwargs_str = ', '.join([f"{k}={repr(v)}" for k, v in kwargs.items()])



        logger.debug(f"Starting {func_name}({args_str}{', ' if args_str and kwargs_str else
''}{kwargs_str})")



        try:

            start_time = time.time()

            result = func(*args, **kwargs)

            elapsed = time.time() - start_time



            logger.info(f"Completed {func_name} in {elapsed:.3f}s")

            return result



        except Exception as e:

            logger.error(f"Error in {func_name}: {type(e).__name__}: {str(e)}")
```

```
            raise



    return wrapper


# ===== Caching Decorator =====

def cached(ttl_seconds: int = 300, key_prefix: str = ""):

    """

    Decorator to cache function results.

    """

    def decorator(func: Callable) -> Callable:

        @functools.wraps(func)

        def wrapper(*args, **kwargs):

            # Generate cache key

            key_parts = [key_prefix, func.__name__]
```

```
            # Add positional arguments (skip self)
            for arg in args[1:]:
                key_parts.append(str(arg))

            # Add keyword arguments
            for k, v in sorted(kwargs.items()):
                key_parts.append(f"{k}:{v}")

            cache_key = hashlib.md5('|'.join(key_parts).encode()).hexdigest()

            # Check cache
            cached_result = cache.get(cache_key)
            if cached_result is not None:
                logger.debug(f"Cache hit for {func.__name__}")
                return cached_result
```

```
            # Execute function
            logger.debug(f"Cache miss for {func.__name__}")
            result = func(*args, **kwargs)

            # Store in cache
            cache.set(cache_key, result, ttl_seconds)

            return result

        return wrapper
    return decorator

# ===== Retry Decorator =====
def retry(max_attempts: int = 3, delay: float = 1.0, exceptions: tuple = (Exception,)):
```

```
    """
    Decorator to retry a function on failure.
    """
    def decorator(func: Callable) -> Callable:
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            last_exception = None
            for attempt in range(max_attempts):
                try:
                    return func(*args, **kwargs)
                except exceptions as e:
                    last_exception = e
                    if attempt < max_attempts - 1:
                        wait_time = delay * (2 ** attempt)  # Exponential backoff
```

```
                        logger.warning(f"Attempt {attempt + 1} failed for {func.__name__}: {e}.
Retrying in {wait_time:.2f}s")
                        time.sleep(wait_time)

            logger.error(f"All {max_attempts} attempts failed for {func.__name__}")
            raise last_exception

        return wrapper
    return decorator

# ===== Performance Monitoring Decorator =====
def monitor_performance(threshold: float = 1.0):
    """
    Decorator to monitor performance and warn if execution exceeds threshold.
    """
    def decorator(func: Callable) -> Callable:
```

```
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            start_time = time.time()
            result = func(*args, **kwargs)
            elapsed = time.time() - start_time

            if elapsed > threshold:
                logger.warning(f"Performance warning: {func.__name__} took {elapsed:.3f}s (threshold:
{threshold}s)")

            return result

        return wrapper
    return decorator
```

5. External API Client (`external.py`)

```python
# app/external.py

"""

External API client for product data enrichment.

"""

import json

import httpx

from typing import Optional, Dict, Any, List

import asyncio

import logging

from app.decorators import retry, log_execution, cached

logger = logging.getLogger(__name__)


class ExternalAPIClient:
```

```
    """Client for external product data API."""



    def __init__(self, base_url: str, api_key: str):

        self.base_url = base_url.rstrip('/')

        self.api_key = api_key

        self.timeout = 10.0

        self.client = None



    async def _get_client(self) -> httpx.AsyncClient:

        """Get or create HTTP client."""

        if self.client is None:

            self.client = httpx.AsyncClient(

                timeout=self.timeout,

                headers={

                    "Authorization": f"Bearer {self.api_key}",
```

```
                "Content-Type": "application/json"
            }
        )
    return self.client


@retry(max_attempts=3, delay=0.5)
@log_execution
@cached(ttl_seconds=300, key_prefix="external")
async def get_product_data(self, product_id: str) -> Optional[Dict[str, Any]]:
    """
    Fetch product data from external API.
    """
    client = await self._get_client()
    url = f"{self.base_url}/products/{product_id}"
```

```
    logger.debug(f"Fetching external data for product {product_id}")



    try:
        response = await client.get(url)
        response.raise_for_status()
        data = response.json()
        logger.info(f"Successfully fetched data for product {product_id}")
        return data
    except httpx.HTTPStatusError as e:
        logger.error(f"HTTP error fetching product {product_id}: {e.response.status_code}")
        raise
    except Exception as e:
        logger.error(f"Error fetching product {product_id}: {e}")
        raise
```

```python
@log_execution
async def enrich_multiple_products(
    self,
    product_ids: List[str]
) -> Dict[str, Optional[Dict[str, Any]]]:
    """
    Fetch data for multiple products concurrently.
    """
    results = {}

    # Create tasks for all products
    tasks = {
        product_id: self.get_product_data(product_id)
```

```
            for product_id in product_ids
        }

        # Execute all tasks concurrently
        for product_id, task in tasks.items():
            try:
                results[product_id] = await task
            except Exception as e:
                logger.error(f"Failed to enrich product {product_id}: {e}")
                results[product_id] = None

        return results

    async def close(self):
        """Close the HTTP client."""
```

```
        if self.client:
            await self.client.aclose()
            self.client = None

# Global client instance (initialized in main)
external_client: Optional[ExternalAPIClient] = None

async def get_external_client() -> ExternalAPIClient:
    """Dependency for external client."""
    if external_client is None:
        raise RuntimeError("External client not initialized")
    return external_client
```

---

6. CRUD Operations (`crud.py`)

```python
# app/crud.py

"""

CRUD operations for products.

"""

from sqlalchemy.orm import Session

from sqlalchemy import or_, and_, func

from typing import Optional, List, Dict, Any

import logging

from app.models import Product

from app.schemas import ProductCreate, ProductUpdate

from app.decorators import log_execution

logger = logging.getLogger(__name__)


@log_execution
```

```
def get_product(db: Session, product_id: int) -> Optional[Product]:

    """Get a product by ID."""

    return db.query(Product).filter(Product.id == product_id).first()


@log_execution

def get_product_by_sku(db: Session, sku: str) -> Optional[Product]:

    """Get a product by SKU."""

    return db.query(Product).filter(Product.sku == sku.upper()).first()


@log_execution

def get_products(

    db: Session,

    skip: int = 0,

    limit: int = 100,

    search: Optional[str] = None,

    category: Optional[str] = None,
```

```
    min_price: Optional[float] = None,
    max_price: Optional[float] = None,
    in_stock: Optional[bool] = None,
    is_active: Optional[bool] = True
) -> tuple[List[Product], int]:
    """
    Get products with filtering and pagination.
    Returns (products, total_count).
    """
    query = db.query(Product)

    # Apply filters
    if is_active is not None:
        query = query.filter(Product.is_active == is_active)
```

```
if search:
    query = query.filter(
        or_(
            Product.name.ilike(f"%{search}%"),
            Product.sku.ilike(f"%{search}%"),
            Product.description.ilike(f"%{search}%")
        )
    )

if category:
    query = query.filter(Product.category == category)

if min_price is not None:
    query = query.filter(Product.price >= min_price)
```

```python
if max_price is not None:
    query = query.filter(Product.price <= max_price)

if in_stock is not None:
    if in_stock:
        query = query.filter(Product.stock > 0)
    else:
        query = query.filter(Product.stock == 0)

# Get total count
total = query.count()

# Apply pagination and ordering
```

```
    products = query.order_by(Product.id).offset(skip).limit(limit).all()



    return products, total


@log_execution
def create_product(db: Session, product_data: ProductCreate) -> Product:
    """Create a new product."""
    # Check for duplicate SKU
    existing = get_product_by_sku(db, product_data.sku)
    if existing:
        raise ValueError(f"Product with SKU {product_data.sku} already exists")



    db_product = Product(**product_data.model_dump())
    db.add(db_product)
    db.commit()
```

```
    db.refresh(db_product)



    logger.info(f"Created product: {db_product.id} - {db_product.name}")

    return db_product


@log_execution

def update_product(

    db: Session,

    product_id: int,

    product_data: ProductUpdate

) -> Optional[Product]:

    """Update a product."""

    db_product = get_product(db, product_id)

    if not db_product:

        return None
```

```python
# Check SKU uniqueness if updating SKU
if product_data.sku is not None:
    existing = get_product_by_sku(db, product_data.sku)
    if existing and existing.id != product_id:
        raise ValueError(f"Product with SKU {product_data.sku} already exists")

# Update fields
update_data = product_data.model_dump(exclude_unset=True)
for field, value in update_data.items():
    setattr(db_product, field, value)

db.commit()
db.refresh(db_product)
```

```
    logger.info(f"Updated product: {db_product.id} - {db_product.name}")
    return db_product

@log_execution
def delete_product(db: Session, product_id: int) -> bool:
    """Delete a product (hard delete)."""
    db_product = get_product(db, product_id)
    if not db_product:
        return False

    db.delete(db_product)
    db.commit()

    logger.info(f"Deleted product: {product_id}")
    return True
```

```python
@log_execution
def update_product_stock(db: Session, product_id: int, quantity: int) -> Optional[Product]:
    """Update product stock (add or subtract)."""
    db_product = get_product(db, product_id)
    if not db_product:
        return None

    new_stock = db_product.stock + quantity
    if new_stock < 0:
        raise ValueError(f"Not enough stock. Current: {db_product.stock}, Requested: {-quantity}")

    db_product.stock = new_stock
    db.commit()
    db.refresh(db_product)
```

```python
    logger.info(f"Updated stock for product {product_id}: {new_stock}")
    return db_product

@log_execution
def get_categories(db: Session) -> List[str]:
    """Get all unique categories."""
    results = db.query(Product.category).distinct().filter(
        Product.category.isnot(None)
    ).all()
    return [r[0] for r in results if r[0]]

@log_execution
def get_inventory_stats(db: Session) -> Dict[str, Any]:
    """Get inventory statistics."""
```

```
    total_products = db.query(Product).count()
    active_products = db.query(Product).filter(Product.is_active == True).count()
    total_stock = db.query(func.sum(Product.stock)).scalar() or 0
    low_stock = db.query(Product).filter(
        Product.is_active == True,
        Product.stock > 0,
        Product.stock <= 10
    ).count()
    out_of_stock = db.query(Product).filter(
        Product.is_active == True,
        Product.stock == 0
    ).count()


    return {
        "total_products": total_products,
```

```
        "active_products": active_products,
        "total_stock": total_stock,
        "low_stock": low_stock,
        "out_of_stock": out_of_stock
    }
```

---

7. Dependencies (`dependencies.py`)

```
# app/dependencies.py
"""
FastAPI dependencies.
"""
from typing import Optional, List
from fastapi import Depends, Query, HTTPException, status
```

```
from sqlalchemy.orm import Session

from app.database import get_db

from app.crud import get_product, get_product_by_sku


def get_product_by_id(

    product_id: int,

    db: Session = Depends(get_db)

):

    """

    Dependency to get a product by ID.

    Raises 404 if not found.

    """

    product = get_product(db, product_id)

    if not product:

        raise HTTPException(
```

```
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Product with ID {product_id} not found"
        )
    return product

def get_product_by_sku_or_404(
    sku: str,
    db: Session = Depends(get_db)
):
    """
    Dependency to get a product by SKU.
    Raises 404 if not found.
    """
    product = get_product_by_sku(db, sku)
    if not product:
```

```
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Product with SKU {sku} not found"
        )
    return product

class PaginationParams:
    """
    Dependency for pagination parameters.
    """
    def __init__(
        self,
        page: int = Query(1, ge=1, description="Page number"),
        page_size: int = Query(10, ge=1, le=100, description="Items per page")
    ):
```

```
        self.page = page
        self.page_size = page_size
        self.skip = (page - 1) * page_size


class ProductFilters:
    """
    Dependency for product filter parameters.
    """
    def __init__(
        self,
        search: Optional[str] = Query(None, description="Search by name, SKU, or description"),
        category: Optional[str] = Query(None, description="Filter by category"),
        min_price: Optional[float] = Query(None, ge=0, description="Minimum price"),
        max_price: Optional[float] = Query(None, ge=0, description="Maximum price"),
        in_stock: Optional[bool] = Query(None, description="Filter by availability"),
```

```
        is_active: Optional[bool] = Query(True, description="Filter by active status")

    ):

        self.search = search

        self.category = category

        self.min_price = min_price

        self.max_price = max_price

        self.in_stock = in_stock

        self.is_active = is_active
```

---

8. Router: Products (`routers/products.py`)

```
# app/routers/products.py
"""
Product management endpoints.
"""

from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session
```

```python
from typing import List
import logging

from app.database import get_db
from app.schemas import (
    ProductCreate,
    ProductUpdate,
    ProductResponse,
    ProductListResponse
)
from app.crud import (
    create_product,
    get_products,
    get_product,
    update_product,
    delete_product,
    get_categories,
    get_product_by_sku
)
from app.dependencies import (
    get_product_by_id,
    get_product_by_sku_or_404,  # ADDED: Import the missing dependency
    PaginationParams,
    ProductFilters
)
from app.decorators import log_execution, cached

logger = logging.getLogger(__name__)
```

```python
router = APIRouter(prefix="/products", tags=["products"])


@router.get("/", response_model=ProductListResponse)
@log_execution
@cached(ttl_seconds=60, key_prefix="products_list")
def get_products_endpoint(
    pagination: PaginationParams = Depends(),
    filters: ProductFilters = Depends(),
    db: Session = Depends(get_db)
):
    """
    Get all products with filtering and pagination.
    """
    products, total = get_products(
        db,
        skip=pagination.skip,
        limit=pagination.page_size,
        search=filters.search,
        category=filters.category,
        min_price=filters.min_price,
        max_price=filters.max_price,
        in_stock=filters.in_stock,
        is_active=filters.is_active
    )

    return ProductListResponse(
        items=products,
        total=total,
```

```
        page=pagination.page,
        page_size=pagination.page_size,
        pages=(total + pagination.page_size - 1) // pagination.page_size
    )


@router.get("/categories", response_model=List[str])
@log_execution
@cached(ttl_seconds=300, key_prefix="categories")
def get_categories_endpoint(
    db: Session = Depends(get_db)
):
    """Get all unique categories."""
    return get_categories(db)


@router.get("/{product_id}", response_model=ProductResponse)
@log_execution
@cached(ttl_seconds=60, key_prefix="product_detail")
def get_product_endpoint(
    product: ProductResponse = Depends(get_product_by_id)
):
    """
    Get a product by ID.
    """
    return product


@router.get("/sku/{sku}", response_model=ProductResponse)
```

```python
@log_execution
@cached(ttl_seconds=60, key_prefix="product_sku")
def get_product_by_sku_endpoint(
    product: ProductResponse = Depends(get_product_by_sku_or_404)  # Now defined
):
    """
    Get a product by SKU.
    """
    return product


@router.post("/", response_model=ProductResponse, status_code=status.HTTP_201_CREATED)
@log_execution
def create_product_endpoint(
    product_data: ProductCreate,
    db: Session = Depends(get_db)
):
    """
    Create a new product.
    """
    try:
        return create_product(db, product_data)
    except ValueError as e:
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail=str(e)
        )
```

```python
@router.patch("/{product_id}", response_model=ProductResponse)
@log_execution
def update_product_endpoint(
    product_data: ProductUpdate,
    product_id: int,
    db: Session = Depends(get_db),
    existing_product: ProductResponse = Depends(get_product_by_id)
):
    """
    Update a product.
    """
    try:
        product = update_product(db, product_id, product_data)
        if not product:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail=f"Product with ID {product_id} not found"
            )
        return product
    except ValueError as e:
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail=str(e)
        )


@router.delete("/{product_id}", status_code=status.HTTP_204_NO_CONTENT)
@log_execution
def delete_product_endpoint(
```

```
    product_id: int,
    db: Session = Depends(get_db),
    existing_product: ProductResponse = Depends(get_product_by_id)
):
    """
    Delete a product.
    """
    if not delete_product(db, product_id):
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Product with ID {product_id} not found"
        )
```

---

9. Router: Inventory (`routers/inventory.py`)

```
# app/routers/inventory.py

"""

Inventory management endpoints.

"""

from fastapi import APIRouter, Depends, HTTPException, status

from sqlalchemy.orm import Session
```

```python
from typing import List, Dict, Any

import logging

from app.database import get_db

from app.schemas import (

    ProductResponse,

    ProductEnrichmentResponse,

    BatchEnrichmentResponse

)

from app.crud import (

    get_product,

    update_product_stock,

    get_inventory_stats

)

from app.dependencies import get_product_by_id
```

```
from app.external import get_external_client, ExternalAPIClient

from app.decorators import log_execution, monitor_performance

logger = logging.getLogger(__name__)

router = APIRouter(prefix="/inventory", tags=["inventory"])


@router.get("/stats")

@log_execution

@monitor_performance(threshold=0.5)

def get_inventory_stats_endpoint(

    db: Session = Depends(get_db)

):

    """

    Get inventory statistics.

    """

    return get_inventory_stats(db)
```

```
@router.patch("/stock/{product_id}")
@log_execution
def update_stock_endpoint(
    product_id: int,
    quantity: int,
    db: Session = Depends(get_db),
    existing_product: ProductResponse = Depends(get_product_by_id)
):
    """
    Update product stock (positive to add, negative to subtract).
    """
    try:
        product = update_product_stock(db, product_id, quantity)
        return {
```

```
            "id": product.id,
            "name": product.name,
            "stock": product.stock,
            "message": f"Stock updated by {quantity} units"
        }
    except ValueError as e:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail=str(e)
        )

@router.get("/enrich/{product_id}", response_model=ProductEnrichmentResponse)
@log_execution
@monitor_performance(threshold=1.0)
async def enrich_product_endpoint(
```

```
    product_id: int,
    db: Session = Depends(get_db),
    product: ProductResponse = Depends(get_product_by_id),
    external: ExternalAPIClient = Depends(get_external_client)
):
    """
    Enrich a single product with external data.
    """
    # Fetch external data
    try:
        external_data = await external.get_product_data(product.sku)
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
            detail=f"External API error: {str(e)}"
```

```
    )


if not external_data:
    raise HTTPException(
        status_code=status.HTTP_404_NOT_FOUND,
        detail=f"No external data found for SKU {product.sku}"
    )


# Update product with external data
db_product = get_product(db, product_id)
db_product.external_id = external_data.get('id')
db_product.external_data = external_data


db.commit()
```

```
    db.refresh(db_product)



    return ProductEnrichmentResponse(

        id=db_product.id,

        name=db_product.name,

        enriched_data=external_data,

        enriched_at=db_product.updated_at

    )


@router.post("/enrich/batch", response_model=BatchEnrichmentResponse)

@log_execution

@monitor_performance(threshold=2.0)

async def enrich_batch_endpoint(

    product_ids: List[int],

    db: Session = Depends(get_db),
```

```
    external: ExternalAPIClient = Depends(get_external_client)

):

    """

    Enrich multiple products with external data.

    """

    if not product_ids:

        raise HTTPException(

            status_code=status.HTTP_400_BAD_REQUEST,

            detail="Product IDs list cannot be empty"

        )



    # Get products

    products = []

    sku_to_id = {}

    for pid in product_ids:
```

```
        product = get_product(db, pid)
        if product:
            products.append(product)
            sku_to_id[product.sku] = product.id


    if not products:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="No valid products found"
        )


    # Fetch external data
    skus = [p.sku for p in products]
    external_data_map = await external.enrich_multiple_products(skus)
```

```
# Update products
enriched_count = 0
failed_count = 0
details = []


for product in products:
    external_data = external_data_map.get(product.sku)
    if external_data:
        product.external_id = external_data.get('id')
        product.external_data = external_data
        enriched_count += 1
        details.append({
            "product_id": product.id,
            "sku": product.sku,
```

```
                "status": "enriched"
            })
        else:
            failed_count += 1
            details.append({
                "product_id": product.id,
                "sku": product.sku,
                "status": "failed",
                "reason": "No external data found"
            })

    db.commit()

    return BatchEnrichmentResponse(
```

```
        total=len(products),
        enriched=enriched_count,
        failed=failed_count,
        details=details
    )
```

---

10. Main Application (`main.py`)

```
# app/main.py
"""
Main FastAPI application.
"""
from fastapi import FastAPI, HTTPException, status
from fastapi.responses import JSONResponse
from fastapi.middleware.cors import CORSMiddleware
```

```python
import logging

from contextlib import asynccontextmanager

from app.database import engine, Base

from app.routers import products, inventory

from app.external import ExternalAPIClient, external_client

from app.decorators import cache

# Configure logging

logging.basicConfig(

    level=logging.INFO,

    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'

)

logger = logging.getLogger(__name__)


@asynccontextmanager

async def lifespan(app: FastAPI):
```

```
    """
    Lifespan context manager for startup and shutdown events.
    """
    # Startup
    logger.info("Starting Inventory Management Platform...")


    # Create database tables
    Base.metadata.create_all(bind=engine)
    logger.info("Database tables created")


    # Initialize external client
    import os
    from dotenv import load_dotenv
    load_dotenv()
```

```
    global external_client

    external_client = ExternalAPIClient(

        base_url=os.getenv("EXTERNAL_API_URL", "https://api.example.com"),

        api_key=os.getenv("EXTERNAL_API_KEY", "test-key")

    )

    logger.info("External API client initialized")



    yield



    # Shutdown

    logger.info("Shutting down Inventory Management Platform...")

    if external_client:

        await external_client.close()

    cache.clear()
```

```
    logger.info("Cleanup complete")


# Create FastAPI application

app = FastAPI(

    title="Inventory Management Platform",

    description="Complete inventory management system with external data enrichment",

    version="1.0.0",

    lifespan=lifespan

)

# Configure CORS

app.add_middleware(

    CORSMiddleware,

    allow_origins=["*"],

    allow_credentials=True,

    allow_methods=["*"],
```

```
    allow_headers=["*"],

)

# Include routers

app.include_router(products.router)

app.include_router(inventory.router)


# ===== Health Check =====

@app.get("/health")

async def health_check():

    """Health check endpoint."""

    return {

        "status": "healthy",

        "service": "inventory-platform",

        "version": "1.0.0"

    }
```

```python
# ===== Global Exception Handlers =====

@app.exception_handler(HTTPException)

async def http_exception_handler(request, exc):

    """Handle HTTP exceptions."""

    logger.warning(f"HTTP Exception: {exc.status_code} - {exc.detail}")

    return JSONResponse(

        status_code=exc.status_code,

        content={

            "error": True,

            "status_code": exc.status_code,

            "detail": exc.detail

        }

    )


@app.exception_handler(Exception)
```

```python
async def general_exception_handler(request, exc):
    """Handle all other exceptions."""
    logger.error(f"Unhandled exception: {type(exc).__name__}: {str(exc)}")
    return JSONResponse(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        content={
            "error": True,
            "status_code": 500,
            "detail": "An internal error occurred"
        }
    )


# ===== Root Endpoint =====
@app.get("/")
async def root():
```

```
    """Root endpoint with API information."""
    return {
        "service": "Inventory Management Platform",
        "version": "1.0.0",
        "endpoints": {
            "products": "/products",
            "inventory": "/inventory",
            "docs": "/docs",
            "health": "/health"
        }
    }
```

---

11. Tests (`tests/test_products.py`)

```python
# tests/test_products.py

"""

Tests for product endpoints.

"""

import pytest

from fastapi.testclient import TestClient

from sqlalchemy import create_engine

from sqlalchemy.orm import sessionmaker

from app.main import app

from app.database import Base, get_db

from app.models import Product

# Test database

SQLALCHEMY_DATABASE_URL = "sqlite:///./test.db"

engine = create_engine(SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread": False})
```

```
TestingSessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)


def override_get_db():
    """Override database dependency for testing."""
    try:
        db = TestingSessionLocal()
        yield db
    finally:
        db.close()


app.dependency_overrides[get_db] = override_get_db
client = TestClient(app)


@pytest.fixture(autouse=True)
def setup_database():
    """Setup test database."""
```

```
    Base.metadata.create_all(bind=engine)
    yield
    Base.metadata.drop_all(bind=engine)


@pytest.fixture
def sample_product():
    """Create a sample product for testing."""
    return {
        "name": "Test Product",
        "sku": "TEST001",
        "price": 99.99,
        "stock": 10,
        "category": "Electronics",
        "description": "Test description"
    }
```

```python
def test_create_product(sample_product):
    """Test creating a product."""
    response = client.post("/products", json=sample_product)
    assert response.status_code == 201
    data = response.json()
    assert data["name"] == sample_product["name"]
    assert data["sku"] == sample_product["sku"]
    assert data["price"] == sample_product["price"]
    assert data["id"] is not None


def test_create_duplicate_product(sample_product):
    """Test creating a product with duplicate SKU."""
    client.post("/products", json=sample_product)
    response = client.post("/products", json=sample_product)
```

```python
    assert response.status_code == 409


def test_get_products(sample_product):
    """Test getting products list."""
    client.post("/products", json=sample_product)



    response = client.get("/products")
    assert response.status_code == 200
    data = response.json()
    assert data["total"] >= 1
    assert len(data["items"]) >= 1


def test_get_product_by_id(sample_product):
    """Test getting a product by ID."""
    create_response = client.post("/products", json=sample_product)
    product_id = create_response.json()["id"]
```

```
    response = client.get(f"/products/{product_id}")
    assert response.status_code == 200
    data = response.json()
    assert data["id"] == product_id
    assert data["name"] == sample_product["name"]


def test_get_product_not_found():
    """Test getting a non-existent product."""
    response = client.get("/products/99999")
    assert response.status_code == 404


def test_update_product(sample_product):
    """Test updating a product."""
    create_response = client.post("/products", json=sample_product)
```

```python
    product_id = create_response.json()["id"]



    update_data = {"name": "Updated Product", "price": 149.99}

    response = client.patch(f"/products/{product_id}", json=update_data)



    assert response.status_code == 200

    data = response.json()

    assert data["name"] == "Updated Product"

    assert data["price"] == 149.99


def test_delete_product(sample_product):

    """Test deleting a product."""

    create_response = client.post("/products", json=sample_product)

    product_id = create_response.json()["id"]
```

```python
    response = client.delete(f"/products/{product_id}")
    assert response.status_code == 204

    # Verify product is deleted
    get_response = client.get(f"/products/{product_id}")
    assert get_response.status_code == 404

def test_get_categories(sample_product):
    """Test getting categories."""
    client.post("/products", json=sample_product)

    response = client.get("/products/categories")
    assert response.status_code == 200
    data = response.json()
    assert "Electronics" in data
```

```python
def test_products_pagination(sample_product):
    """Test product pagination."""
    # Create multiple products
    for i in range(15):
        product = sample_product.copy()
        product["sku"] = f"TEST{i:03d}"
        product["name"] = f"Test Product {i}"
        client.post("/products", json=product)


    # Test first page
    response = client.get("/products?page=1&page_size=10")
    assert response.status_code == 200
    data = response.json()
    assert len(data["items"]) == 10
```

```
    assert data["total"] >= 15
    assert data["pages"] >= 2

def test_product_filtering(sample_product):
    """Test product filtering."""
    client.post("/products", json=sample_product)


    # Filter by category
    response = client.get("/products?category=Electronics")
    assert response.status_code == 200
    data = response.json()
    assert len(data["items"]) >= 1


    # Filter by search
    response = client.get("/products?search=Test")
```

```
    assert response.status_code == 200

    data = response.json()

    assert len(data["items"]) >= 1
```

---

12. Tests for Inventory (`tests/test_inventory.py`)

```
# app/dependencies.py
"""
Dependency injection for FastAPI endpoints.
"""

from fastapi import Depends, HTTPException, status
from sqlalchemy.orm import Session
from typing import Optional

from app.database import get_db
from app.crud import get_product, get_product_by_sku
from app.schemas import ProductResponse


def get_product_by_id(
    product_id: int,
```

```
    db: Session = Depends(get_db)
) -> ProductResponse:
    """
    Dependency that retrieves a product by ID.
    Raises 404 if not found.
    """
    product = get_product(db, product_id)
    if not product:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Product with ID {product_id} not found"
        )
    return product


def get_product_by_sku_or_404(
    sku: str,
    db: Session = Depends(get_db)
) -> ProductResponse:
    """
    Dependency that retrieves a product by SKU.
    Raises 404 if not found.
    """
    product = get_product_by_sku(db, sku)
    if not product:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Product with SKU {sku} not found"
        )
```

```python
    return product


class PaginationParams:
    """
    Dependency for pagination parameters.
    """
    def __init__(
        self,
        page: int = 1,
        page_size: int = 20
    ):
        self.page = max(1, page)
        self.page_size = min(100, max(1, page_size))
        self.skip = (self.page - 1) * self.page_size


class ProductFilters:
    """
    Dependency for product filters.
    """
    def __init__(
        self,
        search: Optional[str] = None,
        category: Optional[str] = None,
        min_price: Optional[float] = None,
        max_price: Optional[float] = None,
        in_stock: Optional[bool] = None,
        is_active: Optional[bool] = None
```

```
    ):
        self.search = search
        self.category = category
        self.min_price = min_price
        self.max_price = max_price
        self.in_stock = in_stock
        self.is_active = is_active
```

Running the Application

1. Installation

Create virtual environment

python -m venv venv

source venv/bin/activate # On Windows: venv\Scripts\activate `venv\Scripts\activate.bat`
`venv\Scripts\activate.bat`

Install dependencies

pip install -r requirements.txt

If the above fails:
python -m pip install --upgrade pip setuptools wheel
pip install pydantic>=2.5.0

```
# Create .env file (not required as the .env is already created)

echo "DATABASE_URL=sqlite:///./inventory.db" > .env

echo "EXTERNAL_API_URL=https://fakestoreapi.com/products" >> .env

echo "EXTERNAL_API_KEY=demo-key-not-required" >> .env
```

2. Running the Server

```
# Run with uvicorn

uvicorn app.main:app --reload --host 127.0.0.1 --port 8000

# Or run directly

python -m app.main
```

3. Running Tests

```
# Run all tests

pytest tests/ -v

# Run specific test file

pytest tests/test_products.py -v
```

4. Accessing the API

- **API Documentation**: http://localhost:8000/docs
- **Alternative Docs**: http://localhost:8000/redoc
- **Health Check**: http://localhost:8000/health

API Endpoints Summary

Method	Endpoint	Description
GET	`/`	API information
GET	`/health`	Health check
GET	`/products`	List products (paginated, filterable)
GET	`/products/categories`	Get all categories
GET	`/products/{product_id}`	Get product by ID
GET	`/products/sku/{sku}`	Get product by SKU
POST	`/products`	Create new product
PATCH	`/products/{product_id}`	Update product
DELETE	`/products/{product_id}`	Delete product
GET	`/inventory/stats`	Get inventory statistics
PATCH	`/inventory/stock/{product_id}`	Update product stock
GET	`/inventory/enrich/{product_id}`	Enrich product with external data
POST	`/inventory/enrich/batch`	Enrich multiple products

Key Features Demonstrated

1. **FastAPI Backend**

- Automatic OpenAPI documentation
- Type hints and validation with Pydantic
- Dependency injection

2. **SQLAlchemy ORM**

- Database models with relationships
- CRUD operations
- Session management

3. **Async External Data Enrichment**

- Async HTTP client with httpx
- Concurrent requests
- Error handling and retries

4. **Custom Decorators**

- `@log_execution`: Logging function calls
- `@cached`: In-memory caching with TTL
- `@retry`: Automatic retry on failure
- `@monitor_performance`: Performance monitoring

5. **Testing**

- Unit tests with pytest
- Test database isolation
- Mocking external dependencies

6. **Logging**

- Structured logging with different levels
- File and console handlers
- Contextual information

Testing Using Swagger

1. Start the server Run:

bash uvicorn app.main:app --reload --host 127.0.0.1 --port 8000 Wait until you see:

text Uvicorn running on http://127.0.0.1:8000 2. Open Swagger UI In your browser, go to:

text http://127.0.0.1:8000/docs You will see all endpoints grouped by tags: products and inventory.

3. Test basic endpoints Health check GET /health

Click Try it out → Execute.

Expect:

json { "status": "healthy", "service": "inventory-platform", "version": "1.0.0" } Root info GET /

Execute.

Expect API info with available endpoints.

4. Test product endpoints Create a product POST /products/

Click Try it out.

Send:

json { "name": "Laptop", "description": "Business laptop", "price": 850.0, "stock": 20, "category": "Electronics", "sku": "LAPTOP001" }
Execute.

Expect 201 Created with product details.

List products GET /products/

Try with query params:

page=1

page_size=10

category=Electronics

Execute.

Expect a paginated list.

Get one product GET /products/{product_id}

Use the id from the create response.

Execute.

Update product PATCH /products/{product_id}

Send:

json { "price": 799.0, "stock": 15 } Execute.

Delete product DELETE /products/{product_id}

Execute.

Expect 204 No Content.

5. Test inventory endpoints Inventory stats GET /inventory/stats

Execute.

Expect stats like total products, total stock, low stock, etc.

Update stock PATCH /inventory/stock/{product_id}

Query param: quantity=5 (or -3 to reduce).

Execute.

Enrich one product GET /inventory/enrich/{product_id}

Execute.

This calls the external API and stores external data.

Batch enrich POST /inventory/enrich/batch

Send:

json [1, 2, 3] Execute.

6. Alternative docs You can also use:

text http://127.0.0.1:8000/redoc for a different documentation UI.

Project Explanation

Project gist

This project is an **Inventory Management Platform** built with FastAPI, SQLAlchemy, Pydantic, `httpx`, custom decorators, and pytest.

Its main responsibilities are:

- Store product information in a SQLite database.
- Create, read, update, and delete products.
- Search and filter products.
- Paginate product results.
- Track stock levels and inventory statistics.
- Enrich local product data using an external API.
- Cache repeated results.
- Retry failed external API calls.
- Log function execution and performance.
- Validate request and response data.
- Test product and inventory functionality.

The application follows a layered architecture:

Client

↓

FastAPI routers

↓

Dependencies and validation

↓

CRUD functions

↓

SQLAlchemy models

↓

SQLite database

The external enrichment flow is:

Inventory router

↓

External API client

↓

httpx request

↓

Retry / cache / logging decorators

↓

Product record updated with external data

The uploaded project describes this architecture and its implementation in detail. ppl-ai-file-upload.s3.amazonaws

Project structure

inventory_platform/

├── app/

│ ├── __init__.py

│ ├── main.py

│ ├── models.py

│ ├── schemas.py

│ ├── database.py

│ ├── crud.py

│ ├── dependencies.py

│ ├── decorators.py

│ ├── external.py

│ └── routers/

│ ├── __init__.py

```
│       ├── products.py
│       └── inventory.py
├── tests/
│   ├── __init__.py
│   ├── test_products.py
│   └── test_inventory.py
├── requirements.txt
└── .env
```

Application files

`app/main.py`

This is the **application entry point**.

It:

- Creates the FastAPI application.
- Configures application metadata.
- Configures logging.
- Creates database tables during startup.
- Initializes the external API client.

- Cleans up resources during shutdown.
- Registers the product and inventory routers.
- Configures CORS.
- Defines the root endpoint `/`.
- Defines the health-check endpoint `/health`.
- Defines global exception handlers.

The application is started with:

uvicorn app.main:app --reload

The `app.main:app` notation means:

- `app.main` → import `main.py` from the `app` package.
- `app` → use the FastAPI object named `app`.

app/models.py

This file contains the **SQLAlchemy ORM models**.

The main model is:

class Product(Base):

It maps to the `products` database table.

Important fields include:

- `id`: Primary key.
- `name`: Product name.
- `description`: Optional product description.

- `price`: Product price.
- `stock`: Available inventory quantity.
- `category`: Product category.
- `sku`: Unique stock-keeping unit.
- `is_active`: Whether the product is active.
- `external_id`: ID from the external service.
- `external_data`: External API data stored as JSON text.
- `created_at`: Creation timestamp.
- `updated_at`: Last-update timestamp.

The model represents how product data is stored physically in the database. ppl-ai-file-upload.s3.amazonaws

app/schemas.py

This file contains **Pydantic schemas**.

Schemas control the shape and validation of data entering and leaving the API.

Important schemas include:

Schema	**Purpose**
`ProductBase`	Common product fields and validation rules
`ProductCreate`	Request body for creating a product
`ProductUpdate`	Request body for partially updating a product
`ProductResponse`	Product data returned to clients

Schema	Purpose
`ProductListResponse`	Paginated product-list response
`ProductEnrichmentResponse`	Response for one enriched product
`BatchEnrichmentResponse`	Response for batch enrichment

For example, `ProductBase` validates:

- Product name length.
- Positive price.
- Non-negative stock.
- SKU length.
- SKU formatting.

The SKU validator converts values to uppercase and rejects spaces:

"abc123" → "ABC123"

`ProductResponse` also converts a stored JSON string from `external_data` into a Python dictionary before returning it.

app/database.py

This file handles database configuration and session management.

It:

- Loads environment variables using `python-dotenv`.
- Reads `DATABASE_URL`.
- Creates the SQLAlchemy engine.

- Creates the `SessionLocal` session factory.
- Defines the shared declarative `Base`.
- Provides the `get_db()` dependency.

The `get_db()` function creates a database session for a request and closes it afterward:

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Routers use it through FastAPI dependency injection:

```
db: Session = Depends(get_db)
```

This ensures that database sessions are not left open.

`app/crud.py`

This file contains the **database operations** for products.

CRUD means:

- Create.

- Read.
- Update.
- Delete.

Important functions include:

- `get_product()`: Find a product by ID.
- `get_product_by_sku()`: Find a product by SKU.
- `get_products()`: Retrieve filtered and paginated products.
- `create_product()`: Insert a product.
- `update_product()`: Modify a product.
- `delete_product()`: Remove a product.
- `update_product_stock()`: Add or subtract inventory.
- `get_categories()`: Return unique categories.
- `get_inventory_stats()`: Calculate inventory metrics.

The router should handle HTTP concerns, while this file handles database operations. That separation makes the code easier to test and maintain.

app/dependencies.py

This file contains reusable FastAPI dependencies.

Important dependencies include:

get_product_by_id()

It retrieves a product by ID.

If the product does not exist, it raises a 404 error:

raise HTTPException(

    status_code=404,

    detail=f"Product with ID {product_id} not found"

)

This prevents the same lookup and error-handling code from being repeated in multiple endpoints.

`get_product_by_sku_or_404()`

This performs the same kind of lookup using the product SKU.

`PaginationParams`

This groups pagination query parameters:

page

page_size

It calculates the database offset:

skip = (page - 1) * page_size

`ProductFilters`

This groups filtering options:

search

category

min_price

max_price

in_stock

is_active

These objects keep router functions cleaner.

app/decorators.py

This file contains reusable decorators for cross-cutting concerns.

Cache

This is an in-memory cache with expiration support.

It provides methods such as:

- `get()`
- `set()`
- `clear()`
- `invalidate()`

It stores values temporarily so repeated requests can avoid repeating database or external API work.

@log_execution

This decorator logs:

- Function name.
- Execution success.
- Execution duration.
- Exceptions.

It is applied to CRUD and endpoint functions.

`@cached`

This decorator caches a function result for a specified number of seconds:

@cached(ttl_seconds=60, key_prefix="products_list")

It is used for operations such as product lists, product details, and categories.

`@retry`

This decorator repeats a failed operation:

@retry(max_attempts=3, delay=0.5)

It uses exponential backoff between attempts.

For example:

Attempt 1 → wait 0.5 seconds

Attempt 2 → wait 1.0 second

Attempt 3 → fail

`@monitor_performance`

This measures execution time and logs a warning when a function exceeds a threshold.

`app/external.py`

This file implements the external API integration.

The `ExternalAPIClient` class:

- Stores the external API base URL.
- Stores the API key.
- Creates an asynchronous `httpx.AsyncClient`.
- Sends product-data requests.
- Handles HTTP errors.
- Supports product enrichment.
- Closes the HTTP client during shutdown.

Important methods include:

`get_product_data()`

This fetches external data for one product.

It uses several decorators:

@retry(...)

@log_execution

@cached(...)

async def get_product_data(...):

Therefore, a request can be:

1. Looked up in the cache.
2. Sent to the external API if not cached.
3. Retried if it fails.
4. Logged during execution.

`enrich_multiple_products()`

This fetches external information for multiple products.

The result is a mapping such as:

{

"SKU001": {...},

"SKU002": {...}

}

`get_external_client()`

This exposes the shared external client through FastAPI dependency injection.

`app/routers/__init__.py`

This marks the `routers` directory as a Python package.

It allows the application to import:

from app.routers import products, inventory

It normally does not contain business logic.

app/routers/products.py

This file contains product-management API endpoints.

The router uses the prefix:

/products

and the tag:

products

GET /products/

Returns a paginated and filterable list of products.

Supported query parameters include:

page

page_size

search

category

min_price

max_price

in_stock

is_active

Example:

GET /products/?page=1&page_size=10&category=Electronics

`GET /products/categories`

Returns unique product categories.

Example response:

[

  "Electronics",

  "Books",

  "Furniture"

]

`GET /products/{product_id}`

Returns one product by its database ID.

If it does not exist, the reusable dependency returns a 404 error.

`GET /products/sku/{sku}`

Returns a product by SKU.

Example:

GET /products/sku/LAPTOP001

`POST /products/`

Creates a new product.

Example request:

```
{
  "name": "Laptop",
  "description": "Development laptop",
  "price": 999.99,
  "stock": 10,
  "category": "Electronics",
  "sku": "LAPTOP001"
}
```

The SKU must be unique.

`PATCH /products/{product_id}`

Partially updates a product.

Example request:

{

  "price": 899.99,

  "stock": 8

}

Only supplied fields are updated.

`DELETE /products/{product_id}`

Deletes a product from the database.

app/routers/inventory.py

This file contains inventory-specific endpoints.

The router uses the prefix:

/inventory

and the tag:

inventory

`GET /inventory/stats`

Returns inventory statistics, such as:

- Total number of products.
- Number of active products.
- Total stock.
- Low-stock products.
- Out-of-stock products.

`PATCH /inventory/stock/{product_id}`

Changes product stock.

The `quantity` query parameter can be:

- Positive to add stock.
- Negative to remove stock.

Example:

PATCH /inventory/stock/1?quantity=5

If there is not enough stock for a negative update, it returns an error.

`GET /inventory/enrich/{product_id}`

Fetches external information for one product and stores it in the local product record.

The process is:

1. Find the local product.
2. Send its SKU to the external API.

3. Receive external data.
4. Save the external ID and data.
5. Return the enriched product response.

`POST /inventory/enrich/batch`

Enriches multiple products in one request.

Example request body:

[1, 2, 3]

The response reports:

- Total products processed.
- Successfully enriched products.
- Failed products.
- Details for each product.

Test files

`tests/__init__.py`

Marks the `tests` directory as a Python package.

It normally contains no testing logic.

`tests/test_products.py`

This file tests product endpoints.

It:

- Creates a separate test SQLite database.
- Overrides the normal `get_db()` dependency.
- Creates and drops test tables around tests.
- Uses `TestClient` to make HTTP requests.
- Tests product creation.
- Tests duplicate SKU handling.
- Tests product listing.
- Tests product lookup.
- Tests missing products.
- Tests product updates.
- Tests product deletion.
- Tests categories.
- Tests pagination.
- Tests filtering.

Example test flow:

POST /products

GET /products

PATCH /products/{id}

DELETE /products/{id}

The dependency override is important because tests should not modify the production `inventory.db` database.

`tests/test_inventory.py`

This file tests inventory-specific behavior.

It tests:

- Inventory statistics.
- Adding stock.
- Removing stock.
- Insufficient-stock errors.
- Single-product enrichment.
- Batch enrichment.

The external API is mocked so tests do not depend on the real external service.

For example, `AsyncMock` or `patch` can return fake external data:

```
{

    "id": "ext123",

    "manufacturer": "Test Corp",

    "weight": "2.5kg"

}
```

This makes tests:

- Faster.
- Deterministic.
- Independent of network availability.
- Safe to run repeatedly.

Configuration files

`requirements.txt`

This lists the Python packages required by the project:

Package	**Purpose**
FastAPI	Web API framework
Uvicorn	ASGI server
SQLAlchemy	ORM and database access
Pydantic	Validation and serialization
python-dotenv	Loads `.env` values
httpx	Asynchronous HTTP requests
pytest	Testing framework
pytest-asyncio	Async test support

Install them with:

pip install -r requirements.txt

`.env`

This stores configuration outside the source code:

DATABASE_URL=sqlite:///./inventory.db

EXTERNAL_API_URL=https://api.example.com/products

EXTERNAL_API_KEY=your-api-key

CACHE_TTL=300

LOG_LEVEL=INFO

The main settings are:

- `DATABASE_URL`: Database connection string.
- `EXTERNAL_API_URL`: External product service URL.
- `EXTERNAL_API_KEY`: Credential for the external service.
- `CACHE_TTL`: Cache lifetime.
- `LOG_LEVEL`: Logging level.

In a real project, `.env` should usually be excluded from Git because it may contain secrets.

Main API workflow

Create a product

POST /products/

{

  "name": "Laptop",

  "description": "Business laptop",

```
  "price": 850.0,
  "stock": 20,
  "category": "Electronics",
  "sku": "LAPTOP001"
}
```


List products

GET /products/?page=1&page_size=10

Search products

GET /products/?search=laptop

Filter products

GET /products/?category=Electronics&in_stock=true

Update stock

PATCH /inventory/stock/1?quantity=-2

View inventory statistics

GET /inventory/stats

Enrich a product

GET /inventory/enrich/1

Run tests

pytest tests/ -v

Important implementation observations

The project document contains a few areas that should be corrected before using it as a production-ready project.

Pydantic v2 validator syntax

The documented schemas use older Pydantic v1-style syntax:

from pydantic import validator

and:

@validator("sku")

For Pydantic v2, prefer:

from pydantic import field_validator

and:

@field_validator("sku")

@classmethod

def validate_sku(cls, value):

    ...

Also replace:

class Config:

    from_attributes = True

with:

from pydantic import ConfigDict

model_config = ConfigDict(from_attributes=True)

This is especially relevant because the uploaded project specifies Pydantic 2.5.0. ppl-ai-file-upload.s3.amazonaws

Route ordering

The products router defines:

@router.get("/{product_id}")

before:

@router.get("/sku/{sku}")

A safer approach is to define specific static routes such as `/categories` and `/sku/{sku}` before the general `/{product_id}` route. Otherwise, route matching can become confusing in larger applications.

External client initialization

The `external_client` object is initialized during the application lifespan. This means enrichment endpoints should only be used after application startup has completed.

Caching and database mutations

Product creation, update, and deletion should invalidate related cached results. Otherwise, a cached product list may briefly return old data after a mutation.

Batch enrichment concurrency

The document describes concurrent enrichment, but the shown implementation awaits tasks sequentially. For actual parallel execution, use `asyncio.gather()` with suitable error handling.

One-sentence explanation

This project is a modular FastAPI inventory system where routers expose product and stock APIs, Pydantic validates data, SQLAlchemy persists products, CRUD functions implement database operations, decorators add caching/logging/retry behavior, an async client enriches products from an external service, and pytest verifies the complete workflow.

Here is the very brief flow of the project:

Start server → uvicorn app.main:app creates the FastAPI app, initializes the database, and starts the external API client.

Request arrives → A client hits an endpoint like POST /products or GET /inventory/stats.

Dependency injection → FastAPI injects the database session, pagination params, filters, and external client.

Validation → Pydantic schemas validate the incoming request data (e.g., SKU format, price > 0).

CRUD operation → crud.py functions execute database queries (create, read, update, delete).

External enrichment → For inventory endpoints, external.py fetches data from an external API (with retry, cache, and logging).

Response → The result is serialized by Pydantic and returned as JSON.

Testing → pytest runs tests against a separate test database, mocking external calls.